PGS FIELD MANUAL v0

Protocol-Governed Systems — Cognitive Restoration Manual

Architecture · Governance · Execution · Compiler Doctrine · AI Coding Agent Context

Author: Bhash Ganti (aka Bachi)

© 2026 Bhash Ganti. All rights reserved. Released under the Apache-2.0 License.

Status: Public Reference Artifact

Canonical Repository Github: bachipeachy/pgs_workspace

Reference Implementation: pgs_runtime

Audience: Architects · Senior Engineers · Compiler Engineers · Runtime Engineers · AI Coding Agents · Technical Reviewers · System Maintainers

Philosophy: KISS for Cognitive Processing — rapid context restoration, not exhaustive documentation.

Abstract

In PGS, behavior is declared in governed protocol artifacts, validated through compile-time constitutional enforcement, materialized into deterministic execution topology, and executed by a semantic-agnostic runtime constrained to previously declared admissible behavior. The runtime does not infer permissible behavior dynamically — it traverses topology already declared, validated, sealed, and compiled.

This manual serves two purposes: rapid cognitive restoration for engineers already familiar with the architecture, and high-density context restoration for AI coding assistants performing modification, enhancement, or extension work. It prioritizes doctrine, invariants, governance boundaries, and execution semantics over tutorial material.

PGS is designed for environments where humans and AI systems collaboratively construct software systems while governance integrity remains structurally enforced independent of implementation authorship.

Why This Manual Exists

Architectural intent diffuses into implementation. Behavioral authority becomes hard to locate. Governance becomes retrospective. Onboarding requires reconstructing distributed mental state. AI coding agents compound this — without explicit context they require repeated re-explanation of invariants and doctrine.

This manual is a high-density architectural cognition restoration artifact — not exhaustive documentation.

Objective: Restore the correct architectural mental model in minutes, not hours.

Intended Audience

Primary: System architects · Runtime engineers · Compiler engineers · Governance engineers · AI coding assistants operating under human supervision

Secondary: Security reviewers · Technical maintainers · Researchers studying governed execution systems

---

AI Coding Assistant Usage Doctrine

PGS contains numerous non-conventional constraints that general-purpose coding agents violate unintentionally.

Constraints that violate default coding instincts: - no fallback logic - no runtime discovery - no heuristic resolution - no dynamic imports - no runtime topology synthesis - no short-name artifact resolution - no ambient authority - compile-time governance supremacy - strict separation:
compiler · runtime · governance · topology · authority · transport

Without explicit restoration, generic agents tend to: - introduce convenience abstractions - collapse governance boundaries - embed domain logic into runtime layers - add fallback behavior - infer structure dynamically - weaken determinism

AI Agent Operating Assumption

The agent is assumed to have partial context, lack full architectural memory continuity, optimize locally, and prefer convenience abstractions unless doctrine is explicitly restated.

Therefore: treat this manual as architectural restoration state before any substantial modification.

Critical Reader Guidance

Do not interpret PGS through the lens of: service-oriented architectures · microservice orchestration · middleware routing · policy-after-implementation governance · runtime-discovered plugins · imperative workflow engines · event-driven orchestration fabrics

PGS is **not:** workflow orchestration with governance wrappers · a policy engine · a runtime authorization framework · a service mesh · a BPM engine

PGS **is:** compile-time admissible execution construction

Architectural Reading Rule: When ambiguity exists — governance overrides implementation convenience. This single rule resolves most architectural decisions correctly.

Goals / Non-Goals

Goals - Restore architectural cognition in minutes without re-reading repositories or papers - Prevent architectural drift through concise invariants and doctrine - Serve as context substrate for both engineers and AI coding agents - Unify doctrine and terminology across all PGS repositories - Enable AI coding agents to perform high-fidelity modification without repeated architectural re-explanation

Non-Goals - Not a tutorial — assumes PGS familiarity - Not an implementation reference — see repositories and compiled artifacts - Not exhaustive — implementation detail lives in repos - Not a protocol spec — constitutions and invariants remain authoritative - Not a replacement for source code — repos are the implementation authority - Not a replacement for formal verification or proof systems

---

How to Use This Manual

This manual is optimized for rapid context restoration, not sequential reading. Sections are semantically compressed, context-independent where possible, and architecture-first.

Source code, constitutions, invariants, and protocol artifacts remain authoritative. This manual restores the mental model needed to read them correctly.

Key term: *Admissibility* — behavior permitted by protocol governance to exist and execute. If behavior is not admissible, it is never constructed; it cannot be blocked because it cannot be expressed.

Situation	Recommended Sections
New architectural enhancement	Executive Doctrine · Core Doctrine · Architectural Invariants

Situation	Recommended Sections
Runtime modification	Runtime Model · Execution Model · Anti-Patterns
Compiler work	Compiler Pipeline · Governance Model · Refactor Patterns
Transport changes	Transport Governance · Boundary Orthogonality
AI coding agent onboarding	AI Coding Assistant Usage Doctrine · Executive Doctrine · Anti-Patterns
Governance extension	Federation Boundaries · Invariants · Constitutions
Debugging	Debugging Guide · Trace Lifecycle
New domain integration	Repository Map · Refactor Patterns

---

0. Executive Doctrine

One-line summary: Behavior is a compiled artifact of protocol declarations — not an emergent property of implementation code.

Principle	Statement
Protocol Sovereignty	Protocol is the sole source of behavioral truth
Runtime Dumbness	The runtime has zero domain or business logic knowledge
Compile-Time Resolution	All behavior determined and validated before execution
Zero Inference	Nothing is assumed, guessed, or defaulted
Fail Hard	No fallback, no recovery, no silent failure via assertions
Declaration Over Refactor	Change behavior by changing declarations, not code
Governance Before Execution	Protocol is law; execution is enforcement; trace is proof
Determinism	Identical inputs → identical traces, always
Security by Construction	Unauthorized behavior is never constructed, not merely blocked
Orthogonality	Governance surfaces evolve independently; coupling between surfaces is a violation
Snapshot Sovereignty	Runtime executes compiled state exclusively; no live modification

---

1. Core Doctrine

1.1 Protocol is the Source of Truth

- Behavior identity is carried by protocol artifacts, not code
- Business logic is encoded in protocol artifacts not in implementation code
- Code may be regenerated, replaced, or machine-authored; governance remains stable
- Behavioral authority persists independently of implementation lifecycle

1.2 Runtime Dumbness (Semantic-Agnostic Execution)

- The runtime enforces graph structure — it does not interpret domain meaning
- Same execution engine runs blockchain, AI governance, and collatz conjecture identically
- No domain logic is permitted in the runtime layer

1.3 Compile-Time Resolution

- Execution graphs are constructed before any runtime begins
- Behavioral admissibility is determined at compile time, not runtime
- If the compiler does not construct the path, the implementation cannot traverse it

1.4 Zero Inference

- No implicit defaults, no heuristics, no filesystem scanning
- All artifact locations must be declared explicitly
- No `../` traversal, no `cwd()`, no environment sniffing

1.5 Fail Hard

- Missing artifact → hard failure, not silent skip
- Constraint violation → reject, not log-and-continue
- No graceful degradation that masks architectural violations

1.6 Deterministic Execution

- Execution is a pure function: `Φ(G, input, actor_context) → (result, trace)`
- Replay is structural, not reconstructed

1.7 Governance Before Execution

- Constitutions define what is admissible
- Compiler validates — only conformant behavior enters the snapshot
- Runtime enforces — only snapshot behavior executes

1.8 No Ambient Authority

- Code has no inherent authority derived from its execution context

- All authority originates exclusively from protocol declarations: (AC, IN, WF, CC)
- An operation has exactly the authority declared in its artifacts — no more
- Confused deputy attacks are structurally eliminated, not defended against

1.9 Snapshot Sovereignty

- Runtime executes compiled snapshot state exclusively
- Snapshot is immutable during execution — no live modification
- Runtime behavior cannot diverge from compiled admissibility state
- To change behavior: change protocol source → recompile → rebuild snapshot

1.10 Architectural Compression

- PGS intentionally compresses architecture into a small number of explicit executable concerns
- A smaller ontology with stronger invariants is preferred over a larger ontology with heuristic flexibility
- This reduces governance ambiguity, hidden behavioral surfaces, cognitive reconstruction cost, and AI-agent violation probability
- The closed set of nine execution concerns and five governance surfaces is a feature, not a limitation
- Expanding ontology size without governance necessity is treated as architectural debt

---

2. Architectural Ontology

2.1 The Nine Execution Concerns

Prefix	Name	Role	Group
`TI_`	Transport Ingress	Normalizes external input → canonical internal form	Boundary
`AC_`	Actor Context	Binds execution authority context; establishes admissible principal identity	Authority
`IN_`	Intent	Declarative admission gate; validates payload before graph traversal	Authority
`WF_`	Workflow	Declarative execution topology graph; compile-time governed traversal; determines admissible step sequences	Authority

Prefix	Name	Role	Group
`CC_`	Capability Contract	Named DAG node; declares inputs, outputs, routing outcomes	Capability
`CT_`	Capability Transform	Pure computation; deterministic; zero side effects	Capability
`CS_`	Capability Side Effect	Controlled external state interaction; enumerated, bounded	Capability
`EV_`	Event	Control plane + observability; governance signaling; active, not passive	Observation
`TE_`	Transport Egress	Boundary projection only; formats internal results for external systems	Boundary

Orthogonal Resolution:

Prefix	Name	Role
`RB_`	Runtime Binding	Maps CT_ and CS_ declarations to concrete implementations; provides execution mapping, not authority

2.2 Governance Constructs

Construct	Role
Constitution	Defines invariants for an artifact domain
Invariant	Structural law enforced at compile time
Assertion	Specific check instantiated from an invariant
STRUCTURE	Compiler build configuration; governs scope of compilation
FQDN	`domain::ARTIFACT_CODE_Vn` — canonical artifact identity; no short names
Snapshot	Sealed, read-only compiled output; the runtime's only input
Federation Boundary	Declares a distinct semantic governance authority in `pgs_governance/registry/FB_*/`; one sovereign boundary (FB_CONSTITUTION), all others delegated

Version immutability: Versions are immutable. There is no "latest." A change to behavior requires a new version number (`_V1`, `_V2`, etc.) — the old version remains valid and unchanged in the snapshot.

2.3 Governance Surfaces

PGS has five orthogonal governance surfaces. Each governs a distinct concern. They evolve independently — adding governance sophistication to one surface does not couple

to any other. New gorvernance surface may be added as long separation of concerns principal is not violated by design.

Governance Surface	Governs
Identity	What entities ARE — actor declaration, FQDN, artifact identity
Authority	What entities MAY DO — admissibility, role grants, admission gates
Execution Topology	Admissible traversal — which capabilities execute in what declared sequence; immutable after compilation
Execution	Computation and side-effect semantics — CT purity, CS boundary, output contracts
Boundary	Admission and projection — TI normalization, TE rendering, transport orthogonality

Key invariant: Governance surfaces are orthogonal planes. Topology governs traversal structure only. Authority governs eligibility only. Neither governs the other. Execution is semantically blind to both.

2.4 Artifact Code Prefixes

Prefix	Meaning
`WF_`	Workflow
`CC_`	Capability Contract
`CT_`	Capability Transform
`CS_`	Capability Side Effect
`IN_`	Intent
`RB_`	Runtime Binding
`EV_`	Event
`AC_`	Actor Context
`TI_`	Transport Ingress
`TE_`	Transport Egress
`STRUCTURE_`	Build configuration

---

3. Execution Model

3.1 Governed Execution Topology

PGS execution is **governed declarative graph traversal**, not workflow orchestration. The execution topology is a compile-time governed graph — fully declared before runtime begins, immutable during execution.

```
TI → IN (admission check)
   → WF (governed topology traversal)
     → CC_1 → CT/CS → outcome (SUCCESS | VIOLATION | ALREADY_EXISTS | ...)
     → CC_2 (inputs resolved via JSONPath: $.results.<CC_CODE>.<field>)
     → ...
   → TE (boundary projection)
```

- Topology is a DAG — no cycles, deterministic termination
- All step sequences declared at compile time — runtime is a traversal engine, not an orchestrator
- Edges represent explicit data dependencies and declared control routing
- Outcomes declared in WF_ route traversal — no runtime branching logic
- Runtime receives a fully closed topology graph; it does not discover, repair, or extend it

3.2 CT Purity Invariant

- `Effect(CT) = ∅` — transforms have zero side effects
- CT may call other CTs; may never call CS
- CT correctness is the implementation's responsibility; PGS constrains invocation, not logic

3.3 CS Side Effect Boundary

- CS is the only authorized channel for external state mutation
- `MutationSurface = { s : s ∈ CS_ }`
- No implicit write path exists anywhere else in the system

3.4 Trace as Evidence

- Every node execution emits a structured trace event: artifact identity, inputs, outputs, outcome
- Trace is append-only, immutable once written
- "If an action does not appear in the trace, it is architecturally prohibited"
- Replay is deterministic: same trace → same proof

3.5 Ownership Boundaries

- CC nodes own: input resolution + outcome declaration
- CT implementations own: computation logic within declared contract
- CS implementations own: external IO within declared side-effect scope
- Runtime owns: graph traversal + trace emission
- Compiler owns: admissibility determination

3.6 System Boundary Model

```
External System
      ↓
   TI_  (admission boundary — input normalization only)
      ↓
Execution Graph G  (all admissible execution semantics live here)
      ↓
   TE_  (projection boundary — output rendering only)
      ↓
External System
```

- Execution semantics exist only inside G
- TI_ and TE_ are orthogonal boundary concerns — they do not participate in workflow authority or execution orchestration

3.7 Execution Topology Governance

Doctrine: Execution topology governs traversal structure only.

Topology governance is a first-class constitutional surface. Its invariants are evaluated at compile time; the compiled topology is immutable for the lifetime of the artifact.

Topology Property	Statement
Compile-time governed	All step sequences, routing, and input bindings fully declared before runtime
Declarative	Steps are declared; not inferred, not synthesized, not discovered
Immutable after compilation	Compiled topology cannot be modified, extended, or overridden at runtime
Traversal-only scope	Topology governs which capabilities execute in what sequence — nothing else
Authority orthogonal	Topology may not branch on role, permission, or actor state
Transport orthogonal	Topology may not encode HTTP endpoints, transport conditions, or dispatch logic
Closure required	All step inputs must resolve at compile time; no forward references; no dangling references
No runtime synthesis	Topology may not be generated or inferred from payload content or environment state

Canonical surface governance: Every step's `result_surface` must exactly match the `canonical_surface` declared by the governing `SURFACE_CONTRACT` for that step's capability and operation. Workflow authors MAY route surfaces. Workflow authors MAY NOT define capability semantics.

Topology invariant families (compile-time enforced): - Step identity uniqueness (no duplicate step IDs) - Capability reference uniqueness (exactly one CT or CS per step) - Input reference closure (all `$.results.<step_id>.*` bindings resolve to declared steps) - Routing completeness (every declared surface code has a routing outcome) - Contract closure (CC exit surface exactly matches `result_status_contract.allowed`) - Authority orthogonality (no role-branching, permission routing, or actor-dependent topology) - Transport orthogonality (no HTTP routing, endpoint dispatch, or transport conditions in steps) - Surface canonicality (declared `result_surface` matches `canonical_surface` from SURFACE_CONTRACT)

---

4. Governance Model

4.1 Constitutions

- Each artifact family governed by a constitution
- Constitution declares invariants that all artifacts of that type must satisfy
- Violation → compiler rejects the artifact; it never enters the snapshot

4.2 Invariants

- Structural laws applied at compile time
- Examples: FQDN identity required, no undeclared outcomes, no missing bindings
- Invariants are not advisory — violations are hard failures

4.3 Assertions

- Specific checks instantiated from invariants
- Registered in handler registry; resolved at compile time
- Static imports only — no dynamic loading of assertion handlers

4.4 Protocol Surface Closure (Vocabulary Closure Invariant)

- `Executable Behavior ⊆ Expressible in V` (the declared concern vocabulary)
- Behavior not expressible in V cannot pass governance, cannot enter snapshot, cannot execute
- This is a construction-time property, not a runtime guard

4.5 Governance Dividend

Named Property: As governance surface matures, cost-of-change decreases rather than increases.

- Conventional: low initial governance cost → debt accumulates → per-change cost grows unboundedly

- PGS: higher initial governance cost → debt does not accumulate → per-change cost remains stable
- Complexity: `O(N + M)` vs conventional `O(N × M)` where N = capabilities, M = workflows
- Attack surface: `|CS_| + |AC_| + |RB_|` — finite and enumerable

PGS separates implementation authorship from behavioral admissibility. Behavioral admissibility remains compiler-governed regardless of whether implementation is human-authored or machine-authored. AI may generate implementation, transforms, and workflow proposals — but AI does not define admissibility. Governance defines admissibility. Compiler constructs it. Runtime enforces it. Trace proves it.

4.6 Federation Boundaries

The governance registry in `pgs_governance/registry/` is organized into **federation boundaries** — named semantic governance authorities, not filesystem folders.

Current boundary inventory (as of V0):

Boundary	Level	Governance Scope
`FB_CONSTITUTION`	**Sovereign**	Root protocol semantics, governance meta-rules, artifact identity, FQDN
`FB_TOPOLOGY`	Delegated	Execution topology, WF/CC legality, CT/CS surface closure, routing, binding
`FB_TRANSPORT`	Delegated	Ingress/egress semantics, transport boundary rules, admission
`FB_AUTHORITY`	Delegated	Actor authority, execution admissibility, authority state
`FB_VOCABULARY`	Delegated	Protocol terminology, execution state vocabulary
`FB_CONFORMANCE`	Delegated	Test data, conformance assertion rules
`FB_IDENTITY`	Delegated	Actor identity semantics, identity/authority separation
`FB_BLOCKCHAIN`	Delegated	Blockchain domain build configuration
`FB_AI_GOVERNANCE`	Delegated	AI governance domain build configuration
`FB_EXECUTION_SCHEDULING`	Delegated	Execution scheduling semantics — parallelism, synchronization, deterministic joins
`FB_EXECUTION_PLACEMENT`	Delegated	Execution locality, substrate placement, runtime deployment admissibility

Boundary	Level	Governance Scope
FB_SECURITY_DOMAIN	Delegated	Classification domains, compartmentalization, secure information-flow legality
FB_CRYPTOGRAPHIC_TRUST	Delegated	Snapshot signing, attestation, encryption, trust admissibility

FQDN namespace: All governance artifacts use fb.<boundary_lower>::ARTIFACT_CODE format.

```
fb.constitution::CONSTITUTION_FEDERATION_BOUNDARY_V0
fb.topology::CONSTITUTION_EXECUTION_TOPOLOGY_V0
fb.transport::CONSTITUTION_TRANSPORT_V0
fb.constitution::STRUCTURE_BUILD_PLATFORM_CONFIG_V0
```

Sovereignty model: Exactly one sovereign boundary (FB_CONSTITUTION). All others are delegated — they derive authority from FB_CONSTITUTION and must not exceed that delegation. A second sovereign boundary is a constitutional violation.

Governance locality doctrine: A governance artifact belongs in pgs_governance (central) if and only if it governs all PGS systems universally. Domain-specific governance law belongs in the domain repository. Placing domain-specific governance centrally is a locality violation; placing universal governance locally is also a violation.

Anti-sprawl rule: Federation boundaries must not be created to mirror organizational structure, repository topology, deployment units, runtime packaging, or implementation convenience. A boundary exists only when a distinct semantic governance authority exists. The current set of boundaries is open-ended — new boundaries emerge as distinct governance authorities emerge. This contrasts with Functional Layers and Execution Concerns, which are closed sets.

Authoritative doctrine artifact:
fb.constitution::CONSTITUTION_FEDERATION_BOUNDARY_V0

---

5. Repository Map

Repo	Layer	Responsibility	Depends On
pgs_governance	Governance	Constitutional governance, federated boundaries, structural artifact definitions, invariant enforcement	nothing
pgs_compiler	Compiler	Compiler pipeline, admissibility construction, validation,	pgs_governance

Repo	Layer	Responsibility	Depends On
		conformance generation, protocol tooling	
pgs_transport	Transport	Transport realization boundary; ingress and egress adapters for HTTP and CLI surfaces	nothing
pgs_runtime	Execution	pgs_runtime CLI; deterministic DAG traversal; snapshot loading	pgs_compiler artifacts
pgs_capabilities	Capability Substrate	Shared CT_ and CS_ implementations; reusable capability library	pgs_governanc e, pgs_compiler
pgs_blockchain	Domain	Blockchain workflows: identity, wallet, transaction	pgs_capabilitie s, pgs_runtime
pgs_ai_governance	Domain	AI governance workflows: licensing, agent action, reclaim	pgs_capabilitie s, pgs_runtime
pgs_workspace	Entry Point	Compiled snapshot + operational scripts; public developer entry	all (via editable installs)

Dependency direction: pgs_workspace → domains → capabilities → runtime ← compiler ← governance

Install boundary: All repos installed as editable packages into pgs_workspace/.venv/ via bootstrap_pgs.sh. Do not cross-install or mix environments.

---

6. Compiler Pipeline

6.1 Phases (in order)

The compiler pipeline runs nine named stages (S1–S9). Each is a discrete, ordered phase with its own failure surface.

Stage	Name	What Happens
S1	EXTRACT	Artifact files located via STRUCTURE_ build config; deserialized into typed PIR structures; no filesystem scanning
S2	CANONICALIZE	Artifacts normalized; edge typing resolved; canonical representation established
S3	SEMANTIC_ADDRESSING	Semantic addresses allocated for all graph entities; address space closed
S4	GOVERN	Invariants evaluated via registered assertion handlers; topology closure, step identity, canonical

Stage	Name	What Happens
		surface, authority/transport orthogonality, and graph completeness all checked
S5	CONSTRUCT	Execution topology constructed; CT/CS IR built; CC projections assembled; bindings resolved; topology sealed
S6	PROJECT	Visualization projections generated (graph JSON, PNG rendering)
S7	MATERIALIZE	Compiled artifacts written to snapshot directories; evidence graph written to `evidence_snapshot/`; trace events flushed
S8	VERIFY	All materialized artifacts verified on disk; hash integrity checked; evidence graph integrity validated
S9	ATTEST	Cryptographic attestation computed and written for each structure and the full snapshot

6.2 Compiler Constraints (Non-Negotiable)

- No execution during compilation — compiler never runs CT/CS implementations
- Static imports only — no dynamic module loading
- Deterministic output — same source → same snapshot, always
- No cross-artifact inference — each artifact is validated against its own schema + invariants
- Handler registry must be fully populated before compile starts
- Compiler validates and rejects — it never repairs malformed protocol state

6.3 Build Commands

```
# Phase A — per-structure compilation (run in order)
python -m pgs_compiler.cli compile --structure STRUCTURE_BUILD_PLATFORM_CONFI
G_V0
python -m pgs_compiler.cli compile --structure STRUCTURE_BUILD_BLOCKCHAIN_CON
FIG_V0
python -m pgs_compiler.cli compile --structure STRUCTURE_BUILD_AI_GOVERNANCE_
CONFIG_V0

# Phase B — cross-structure aggregation (run after all Phase A)
python -m pgs_compiler.cli compile --structure STRUCTURE_BUILD_VOCABULARY_AGG
REGATE_V0

# Full build — compile + sync + conformance + attestation + snapshot validati
on
python -m pgs_compiler.cli build --workspace /abs/path/to/pgs_workspace
```

6.4 Compiler Evidence Graph

The compiler emits `evidence_graph.json` per structure during S7. This is the compiler's own governed observability artifact — a semantic causality graph over all S1–S7 trace events, analogous in role to the execution trace for the runtime.

What it contains:

Field	Content
`events`	All compiler trace events (S1–S7), typed by operation, tagged by family
`edges`	CAUSALITY edges + STAGE_SEQUENCE edges
`families`	Events bucketed by semantic concern (DISCOVERY, TOPOLOGY, ADDRESSING, GOVERNANCE, CONSTRUCTION, PROJECTION, MATERIALIZATION)
`evidence_graph_hash`	SHA-256 over core content; verified by S8
`structure_id`, `compiler_version`	Envelope metadata — excluded from hash

Evidence Semantics Doctrine — the formal guarantees:

Guarantee	Statement
Causality definition	Two inferred patterns: (1) S1 `discovery_complete` → all subsequent `node_created` events in the batch; (2) S5 all `ct_ir_built` + `cs_ir_built` + `cc_projection_built` events → `construction_complete`
Stage ordering	Exactly 7 STAGE_SEQUENCE edges (S1→S2→S3→S4→S5→S6→S7); inferred from last event of stage N → first event of stage N+1; deterministic given same source
Event identity	Auto-increment integer, monotonically increasing, immutable per compile run; identical inputs → identical event sequence
Edge kind exhaustiveness	Exactly two edge kinds exist: `CAUSALITY` (parent → child event; event caused/gated target) and `STAGE_SEQUENCE` (last event of stage N → first event of stage N+1). No other edge kinds are permitted. Consumers may assert this.
Edge stability	Edges inferred from operation ordering within the compiled trace batch; same source artifacts always produce the same edge set
Graph determinism	Same source structure artifacts + same compiler version → bitwise-identical `events`, `edges`, `families` content; only

Guarantee	Statement
	envelope fields (`structure_id`, `compiler_version`, `evidence_graph_hash`) may differ between runs
Projection completeness	All events from all stages S1–S7 are present in `evidence_graph.json`; no stage is selectively omitted; every event emitted during compilation appears in the `events` array; all families present in the run appear in the `families` index
Hash contract	SHA-256 over `{event_count, edge_count, events, edges, families}` only; envelope fields (`structure_id`, `compiler_version`, `evidence_graph_hash`) excluded so transport metadata can evolve while graph semantics remain stable; verified by S8 Check 7
Consumer contract	`visualization/consumers/` is the ONLY sanctioned interface; consumers must NEVER import from `compiler/graph/*`; JSON schema is the contract; method signatures on `EvidenceQuery` and `EvidenceProjection` are frozen
Replay guarantee	`evidence_graph.json` is self-contained; no compiler state, no compiler imports needed to parse or interpret it; all consumer operations work from the serialized file alone
Ordering guarantee	`events` array in emission order (S1 first, S7 last, within-stage in emission order); `event_id` values monotonically increasing
VERIFICATION absence	S8's `verification_complete` event is always absent from `evidence_graph.json` by design — S7 writes the file before S8 runs; 0 VERIFICATION-family events is correct, not a defect

Location: `evidence_snapshot/<domain>/evidence_graph.json` — written by S7, verified by S8.

Contract freeze: The `EvidenceQuery` public surface is a protocol surface, not a utility library. Additions require explicit versioning. No compiler-internal fields may be exposed. No shortcuts that bypass DTO semantics.

6.5 Two Compiler Phase Types

The compiler pipeline has two explicit phase types:

Phase Type	Name	Semantics
Phase Type A	Per-Structure Local	DISCOVER → MATERIALIZE within a single structure; single-structure semantic closure
Phase Type B	Cross-Structure Aggregation	Consumes declared output surfaces from multiple structures; produces federated governance products

Phase Type B is governed by an aggregation STRUCTURE_ artifact with an `aggregation_type` field (e.g., `STRUCTURE_BUILD_VOCABULARY_AGGREGATE_V0` with `aggregation_type: VOCABULARY`). The CLI routes on this field to the appropriate aggregation runner.

Phase Type B invariants — non-negotiable: - Deterministic, declarative, bounded, structure-declared - No implicit dependency ordering between structures - No graph inference or hidden phase coupling - All contributing source dirs declared explicitly in `artifact_source_dirs` - Must run after all Phase Type A builds that contribute source artifacts

Current Phase Type B families:

Aggregate	`aggregation_type`	Status
Vocabulary	`VOCABULARY`	Implemented
Transport	`TRANSPORT`	Planned
Authority	`AUTHORITY`	Planned
Conformance	`CONFORMANCE`	Future

---

7. Runtime Model

7.1 Runtime Constraints (Non-Negotiable)

- Snapshot-only: runtime reads `protocol_snapshot/` exclusively; never touches protocol source
- No discovery: artifact locations come from the snapshot manifest, not filesystem scanning
- No inference: missing binding → hard failure; no fallback implementations
- No domain logic: runtime is semantically blind to what it executes
- Deterministic traversal: graph edges are followed exactly as declared

7.2 Runtime Bindings

- `RB_` maps protocol capability declarations to concrete Python implementations
- Both CT_ and CS_ resolved via the same binding mechanism

- RB_ resolves implementation location only — behavioral authority originates exclusively from protocol declarations

7.3 Trace Lifecycle

```
execution start → trace file created at traces/<TRACE_ID>/<TRACE_ID>.jsonl
each CC node    → structured event appended
execution end   → trace sealed; .md summary + .png visualization written
```

Trace files are OUTPUT only. Never use as input to compiler, runtime, or other components.

Trace IDs are deterministic — same inputs produce the same trace ID. Exit codes: `0` for completed execution (including NACK/VIOLATION), `1` for infrastructure failures.

7.3b CS Runtime Types

Six named CS runtime types in the capability substrate:

Runtime Type	Semantics
`RegistryRuntime`	Deduplicated key-value store; ops: REGISTER, RESOLVE, EXISTS, DEREGISTER
`MutableJsonRuntime`	Read-write JSON state; ops: WRITE, READ, DELETE, EXISTS, LIST_KEYS
`AppendOnlyJsonlRuntime`	Immutable event stream; append only; never truncated
`SendEmailRuntime`	Email delivery side effect
`WorkflowGatewayRuntime`	Workflow-to-workflow invocation
`NameRegistryRuntime`	Name-scoped registry variant

7.4 Data State

```
data/
  registry/actors.json          # last-write-wins; deduplicated actor store
  events/identity_events.jsonl  # append-only; never truncated
  governance_actions.json       # AI governance mutable state
  governance_audit.jsonl        # append-only audit stream
  license_facts.json            # read-only seed; never mutated at runtime
```

---

8. Transport Governance

8.1 TI / TE Doctrine

- **TI (Transport Ingress):** Normalizes and validates all external input → canonical internal representation. No business logic.
- **TE (Transport Egress):** Renders internal results for external systems. Boundary projection only. Does not participate in execution semantics.

- Closed-loop invariant: all behavior follows `TI → G → TE`

8.2 Transport Orthogonality

- Transport substrate (CLI, HTTP, queue, agent) is orthogonal to execution semantics
- Changing from CLI to HTTP does not change the execution graph
- No routing logic in TI/TE — routing lives in WF_ declarations

8.3 Transport Does Not Route

- Transport artifacts normalize and project data only
- Transport may not perform execution routing, orchestration, or behavioral selection
- All routing lives in WF_ DAG outcome edges — never in TI/TE middleware

8.4 Snapshot-Driven Route Loading

- HTTP server routes loaded from snapshot; no hardcoded route tables
- Route = (domain, workflow) pair declared in snapshot
- No middleware intelligence — middleware is transport plumbing only

8.5 Transport Federation Doctrine

Transport governance will require Phase Type B federated aggregation to synthesize: - route indices - admission maps - projection schemas - transport boundary manifests

Transport federation **must preserve boundary orthogonality**.

Transport aggregation **may synthesize:** - routes - projections - admission maps

Transport aggregation **may NEVER:** - mutate execution semantics - alter workflows - inject orchestration - infer routing behavior dynamically

This is the “no smart runtime” doctrine expressed at the boundary layer.

One-line doctrine: Transport is not federated execution; it is federated boundary governance.

8.6 HTTP Server

`~/pgs_workspace/scripts/start_http_server.sh`

See `pgs_cli_cheatsheet.txt` for port overrides and manual invocation options.

---

9. Coding Directives

Rule	Rationale
Absolute imports only	No `sys.path` manipulation; no relative import hacks

Rule	Rationale
Layer isolation	CT cannot call CS; runtime cannot contain domain logic; compiler cannot execute
No fallback logic	Missing artifact = failure; no silent defaults
No dynamic discovery	Artifact locations declared, not scanned
No runtime inference	Runtime resolves from snapshot only; never infers from filesystem
No hidden state mutation	All state changes through declared CS_ only
No hardcoded absolute paths	Use script-relative or env-var-driven paths declared explicitly
FQDN everywhere	`domain::ARTIFACT_CODE` — no short names, ever
Static imports only	In operational scripts and compiler handlers
Environment pre-resolved	Runtime and operational environments must be fully resolved before execution begins; no package installation during operation

---

10. Operational Workflows

10.1 Bootstrap (First Time)

```
cd pgs_workspace
cd scripts && ./bootstrap_pgs.sh
source .venv/bin/activate
pgs_runtime --help
```

10.2 Run Demo End-to-End

```
source .venv/bin/activate
cd scripts && ./demo_sample_workflow.sh
```

10.3 Run Workflow (CLI)

```
pgs_runtime run \
  --wf <domain>::<WF_CODE> \
  --payload <path-to-payload.json> \
  --data-root /absolute/path/to/pgs_workspace/data \
  --workspace /absolute/path/to/pgs_workspace
```

Key flags:

Flag	Description
`--wf <FQDN>`	Workflow to run (bypasses explicit intent admission gate)
`--intent <FQDN>`	Alternative to `--wf`; enforces admission gate via declared Intent

Flag	Description
`--payload <file>`	Path to JSON payload file
`--data-root <path>`	**Must be absolute path.** Runtime data root; never inferred
`--workspace <path>`	Workspace root
`--rb <FQDN>`	Override a specific runtime binding (for testing)
`--mode runtime|authoring`	Execution mode (default: authoring)
`--debug`	Enable debug output

Env vars (alternative to flags):

```
export PGS_DATA_ROOT=/absolute/path/to/pgs_workspace/data
export PGS_WORKSPACE=/path/to/pgs_workspace
```

10.4 Examine a Trace

`pgs_runtime examine pgs_workspace/traces/<TRACE_ID>/<TRACE_ID>.jsonl`

10.5 Available Workflow Inventory

All available workflows: `ls protocol_snapshot/artifacts/workflows/`

Domain	Workflow Code	Purpose
blockchain	`WF_REGISTER_ACTOR_UNVERIFIED_V0`	Identity registration
blockchain	`WF_VERIFY_ACTOR_V0`	Identity verification
blockchain	`WF_CREATE_WALLET_V0`	Wallet provisioning
blockchain	`WF_SUBMIT_TRANSACTION_V0`	Transaction submission
ai_governance	`WF_PROVISION_AI_LICENSING_V0`	AI license provisioning
ai_governance	`WF_DENY_PROVISION_V0`	License denial
ai_governance	`WF_AUTO_RECLAIM_V0`	License auto-reclaim
ai_governance	`WF_GOVERN_AGENT_ACTION_V0`	Agent action governance (7 scenarios)
ai_governance	`WF_DEMO_COLLATZ_CONJECTURE_V0`	Domain-neutral execution proof

Payload files: `<repo>/testbed/<subdomain>/test_payloads/`

Pattern (applies to all workflows):

```
pgs_runtime run --wf <domain>::<WF_CODE> \
  --payload <domain_repo>/testbed/<subdomain>/test_payloads/<payload>.json \
  --data-root /absolute/path/to/pgs_workspace/data \
  --workspace /absolute/path/to/pgs_workspace
```

10.6 Unit Tests

```
python pgs_runtime/testbed/run_runtime_tests.py -v
```

10.7 List Available Artifacts

```
ls protocol_snapshot/artifacts/workflows/
ls protocol_snapshot/artifacts/capability_contracts/
ls protocol_snapshot/artifacts/capability_transforms/
ls protocol_snapshot/artifacts/capability_side_effects/
ls protocol_snapshot/artifacts/runtime_bindings/
```

10.8 Clean State Rebuild

```
# Clean compiler outputs
pgs_compiler/scripts/clean_pycache.sh
pgs_compiler/scripts/clean_outputs_dir.sh
pgs_compiler/scripts/clean_compiled_artifacts.sh

# Recompile and sync
python -m pgs_compiler.cli compile --structure STRUCTURE_BUILD_PLATFORM_CONFI
G_V0
python -m pgs_compiler.cli compile --structure STRUCTURE_BUILD_BLOCKCHAIN_CON
FIG_V0
python -m pgs_compiler.cli compile --structure STRUCTURE_BUILD_AI_GOVERNANCE_
CONFIG_V0
python -m pgs_compiler.cli build --workspace /abs/path/to/pgs_workspace
```

---

11. Debugging Guide

11.1 Common Failure Signatures

Symptom	Root Cause	Fix
`ExecutionError: no binding for CT_...`	CT declared in protocol; no RB_ entry	Add RB_ binding for that CT
`ValidationError: payload rejected at IN_...`	Payload violates Intent admission rules	Check IN_ schema; fix payload fields
`BuildError: conformance check failed`	Protocol artifact violates invariant	Run compiler with `--verbose`; read assertion output
`ALREADY_EXISTS` outcome	CS_REGISTRY_V0 found duplicate key	Correct behavior — registry is idempotent by protocol design
Editing `protocol_snapshot/` has no effect	Snapshot is compiled output; runtime reads sealed state	Modify protocol source → recompile → rebuild snapshot

Symptom	Root Cause	Fix
`SnapshotError: artifact not found`	FQDN mismatch or artifact missing from build scope	Check STRUCTURE_ config includes the artifact's domain
Trace shows unexpected routing	Outcome name from CC does not match WF edge	Check CC outcome declarations against WF routing edges
Runtime binding resolution fails	Implementation path changed; RB_ not updated	Update RB_ artifact to point to new implementation

11.2 Compile vs Runtime Failures

Type	When Detected	Meaning
Invariant violation	Compile time	Artifact violates constitutional law — cannot enter snapshot
Schema validation failure	Compile time	Artifact malformed — field missing or wrong type
Missing binding	Runtime startup	Snapshot present but RB_ not wired to implementation
Payload rejection	Runtime, at IN_	Input does not satisfy admission conditions
CC outcome mismatch	Runtime, in DAG	Declared outcome not in WF routing table

11.3 Trace Debugging

```
# Full trace examination
pgs_runtime examine ./traces/<TRACE_ID>/<TRACE_ID>.jsonl

# Each event contains: artifact_id, inputs, outputs, outcome, timestamp
# Read the .md summary for human-readable overview
# Read the .png for visual execution path
```

11.4 Snapshot Mismatch

- If runtime behavior diverges from expected: recompile and sync
- Never patch `protocol_snapshot/` manually — it will be overwritten by next build

---

12. Architectural Invariants

These are hard constraints. Violation = operational failure or architectural corruption.

Invariant	Statement
FQDN Required	All artifact filenames and references must use `domain::ARTIFACT_CODE` — no short names
Snapshot Immutability	`protocol_snapshot/` is READ-ONLY at runtime; never edit by hand
No Runtime Discovery	Runtime resolves artifacts from snapshot manifest only; no filesystem scanning
No Execution in Compiler	Compiler never runs CT/CS/domain implementations
No Transport Execution Logic	TI/TE perform normalization and projection only; no business logic
No Hidden State Mutation	All state changes through explicitly declared CS_ artifacts
No Ambient Authority	Code has no inherent authority; all authority from (AC, IN, WF, CC) declarations
No Fallback	Missing binding, missing artifact, violated invariant → hard failure
Trace Output Only	Traces written to `traces/` by runtime; never used as input to any component
Data Root Explicit	`--data-root` must be explicitly passed; never inferred
Static Imports Only	No dynamic imports in compiler handlers or operational scripts
No sys.path Manipulation	System path must not be modified in scripts or handlers
Protocol Recompile Required	To change behavior → change protocol source → recompile → rebuild snapshot
CT Purity	CT implementations must be free of side effects; no CS calls from CT
Topology Immutability	Compiled execution topology is immutable; no step added, removed, or rerouted at runtime
Topology Closure	All step inputs must resolve at compile time; no dangling references, no forward references
No Runtime Topology Synthesis	Execution topology may not be generated or inferred from payload, environment, or runtime state
Topology Traversal Scope	Execution topology governs traversal structure only; may not encode authority or transport semantics
Canonical Surface	Every step's `result_surface` must exactly match the `canonical_surface` declared in the governing SURFACE_CONTRACT

Invariant	Statement
Evidence Consumer Isolation	`visualization/consumers/` must never import from `compiler/graph/*` or any compiler internal; `evidence_graph.json` JSON schema is the only contract; violation collapses compiler replaceability
Evidence Hash Contract	`evidence_graph_hash` covers only `{event_count, edge_count, events, edges, families}`; envelope fields are excluded; S8 verifies by recomputing over these exact keys
VERIFICATION Family Absent	`evidence_graph.json` covers S1–S7 by design; S8 runs after the file is written; 0 VERIFICATION-family events is architecturally correct — this is not a gap, it is a boundary

---

13. Refactor Patterns

Add a New Artifact Family

1. Define schema in `pgs_governance`
2. Write constitution with invariants
3. Implement assertion handlers (static imports only)
4. Register handlers in handler registry
5. Add STRUCTURE_ config entry to include new family
6. Recompile

Add a New CT (Capability Transform)

1. Implement deterministic function in `pgs_capabilities` library (no side effects)
2. Declare `CT_<NAME>_V0` artifact in the relevant domain repo
3. Add `RB_` binding entry mapping declaration to implementation
4. Reference CT in `CC_` capability contract within a workflow
5. Recompile and sync snapshot

Add a New CS (Capability Side Effect)

1. Implement storage/IO operation in `pgs_capabilities` library
2. Declare `CS_<NAME>_V0` artifact in the relevant domain repo (defines semantics, not implementation)
3. Add `RB_` binding entry
4. Reference CS in `CC_` within a workflow
5. Recompile and sync snapshot

Add a New Domain

1. Create new repo (e.g., `pgs_<domain>`)
2. Author protocol artifacts: `WF_`, `CC_`, `IN_`, `EV_` for the domain
3. Add `STRUCTURE_BUILD_<DOMAIN>_CONFIG_V0` in `pgs_compiler`
4. **Add the domain layer entry in `STRUCTURE_DISCOVERY_V0`** — set `registry_module` to the sub-module path (e.g., `pgs_<domain>.registry`), **not** the top-level package. The compiler's `LayerResolver` uses `module_root.parent` to locate the repo root for compiled output placement; pointing to the top-level package resolves the wrong parent directory and produces zero compiled artifacts.
5. Wire reusable CT/CS from `pgs_capabilities` via `RB_`
6. Implement any domain-specific CT/CS (rarely needed if library is mature)
7. Compile and sync
8. Add domain to workspace bootstrap and HTTP server config

Add a TI Route (HTTP Endpoint)

1. Declare `TI_` artifact in protocol
2. Add static UI payload directory under domain testbed
3. Reference TI in HTTP server `--domain` flag
4. Snapshot-driven route registration happens automatically

Extend STRUCTURE Governance

1. Modify `STRUCTURE_BUILD_*_CONFIG_V0` artifact in `pgs_compiler`
2. Add new artifact directories or scope inclusions
3. Recompile — no runtime changes required

---

14. Anti-Patterns

Anti-Pattern	Why It Violates PGS	Correct Approach
Smart runtime	Embeds domain logic in execution layer	Runtime is generic; all behavior in compiled snapshot
Fallback logic	Masks architectural violations; introduces non-determinism	Hard failure; fix the root cause
Dynamic imports	Breaks static resolution; enables runtime code injection	Static imports only; handler registry at compile time
Filesystem inference	Violates zero-inference doctrine	Declare all paths explicitly
Relative path traversal (`../`)	Path guessing; breaks portability	Script-relative or env-var-driven explicit paths

Anti-Pattern	Why It Violates PGS	Correct Approach
Editing `protocol_snapshot/`	Modifies compiler output; overwritten on next build	Change protocol source → recompile
Using trace as input	Trace is evidence, not protocol; breaks separation	Traces are output only
Hardcoded absolute paths	Environment coupling	Declare via env vars or explicit script parameters
CT calling CS	Breaks CT purity invariant	CS calls only from CC-authorized pathways
Middleware routing logic	Transport coupling; breaks transport orthogonality	Routing in WF_ topology only
Role-branching in topology steps	Embeds authority semantics in execution topology; violates authority orthogonality	Authority evaluation happens before topology traversal; topology assumes admissibility is resolved
`result_surface` deviation from canonical	Workflow author redefines what a capability produces; violates surface contract governance	Declare the exact canonical_surface from the governing SURFACE_CONTRACT — routing is permitted, redefinition is not
Runtime step injection	Synthesizes topology from payload or environment; violates topology immutability	All steps declared at compile time; topology is sealed before runtime begins
Environment-driven branching	Implicit behavior; breaks determinism	Explicit outcomes in protocol artifacts
Short-name artifact references	Breaks FQDN identity; causes resolution failures	Always use `domain::ARTIFACT_CODE`
Package install during operation	Execution environment must be pre-resolved; installing at runtime violates environment isolation	Resolve all dependencies at bootstrap time
`sys.path` manipulation	Breaks import isolation	Proper package installs via editable installs

15. Architecture Evolution Notes

Understanding why the architecture looks as it does prevents re-introducing solved problems.

Evolution	What Changed	Why
Short-name → FQDN	All artifact identifiers now require `domain::CODE` format	Eliminated ambiguity when multiple domains co-exist in snapshot
Runtime decoupling	Runtime became fully domain-agnostic	Enabled same engine to run blockchain and AI governance without modification
Transport governance formalization	TI/TE defined as distinct concern types	Separated projection semantics from execution semantics; enabled transport orthogonality
CT binding refactor	RB_ now resolves both CT_ and CS_ symmetrically	Eliminated split binding mechanism; runtime is fully implementation-agnostic
Snapshot sovereignty	`protocol_snapshot/` declared formally read-only	Prevented drift between compiled and live behavior
TE ontology correction	TE confirmed as boundary projection only; not an execution concern	Clarified that TE has no authority in execution; projection is structural, not behavioral
Federated governance compilation	Compiler gained Phase Type B (cross-structure aggregation); vocabulary was first federated governance product	Single-structure compilation cannot close over cross-structure semantics; vocabulary, transport, and authority require multi-domain synthesis
Protocol-governed execution → federated governance compilation	Architecture class elevated: PGS now governs ontology, routing, and admissibility as compiled artifacts, not runtime concerns	Governance surfaces compound without runtime coupling; execution remains semantically blind while governance grows richer
Execution topology formalized as governed surface	Execution graph structure elevated from implementation convention to constitutional governance surface with 11 enforced invariants	Topology closure, step identity, canonical surface, authority/transport orthogonality, and runtime synthesis prohibition are now compile-time enforced — runtime is provably a traversal engine, not an orchestrator

Evolution	What Changed	Why
Governed declarative graph traversal	PGS execution is no longer adequately described as "workflow orchestration" — it is governed declarative graph traversal	Topology is compile-time declared, authority-orthogonal, transport-orthogonal, and immutable after compilation; these properties are now constitutional, not conventional
Constitutional Federation	Governance registry organized as `registry/FB_*/` federation boundaries (9 boundaries); FQDN namespace is `fb.*::`; STRUCTURE_DISCOVERY_V0 drives all layer resolution; `CONSTITUTION_FEDERATION_BOUNDARY_V0` is the authoritative doctrine artifact; `load_bootstrap_artifact()` requires full FQDN — no fallback	Federation boundaries are first-class semantic constructs. Governance sovereignty is physically visible in the registry. `fb.*::` is the only valid namespace for governance artifacts.
Compiler evidence graph (EG-1→EG-4)	Compiler gained a governed observability artifact: `evidence_graph.json` per structure; events as typed graph nodes (8 families, 19 operations); CAUSALITY + STAGE_SEQUENCE typed edges; SHA-256 hash integrity; verified by S8 Check 7; `visualization/consumers/` consumer contract isolates all downstream consumers from compiler internals	Compiler is now self-observable. The evidence graph enables visualization, AI tooling, replay, and debugging from a stable, compiler-agnostic JSON schema — without importing compiler internals.
Governed Evidence System convergence	PGS architecture converges on three compounding properties: Protocol Compiler + Governed Evidence System + Deterministic Execution Fabric. Each property was independently designed; the convergence is emergent. Most systems solve one.	This combination is an unusual architectural category. The cost-of-change decreasing with system growth is now repeatedly demonstrated architectural behavior, not an aspirational claim.

16. Federated Governance Domains

Architectural transition: PGS has evolved from *protocol-governed execution* to *federated governance compilation*.

This is a substantially more powerful architecture class.

16.1 Definition

A **Federated Governance Domain** is a cross-structure governance concern that: - cannot be semantically closed within a single domain structure - requires contribution from multiple domain builds - is compiled via Phase Type B aggregation - produces a governed artifact consumed by execution, transport, or runtime

16.2 Canonical Domain Families

Domain Type	Example	Product	What It Governs
Federated Aggregation Domain	Vocabulary	`vocabulary_symbols.json` + semantic index	Protocol ontology; expressible concern surface
Federated Boundary Domain	Transport	route index, admission maps, projection schemas	Boundary admission and projection topology
Federated Admissibility Domain	Authority	admissibility maps, actor eligibility graphs	Execution eligibility and observation rights
Federated Correctness Domain	Conformance (future)	global correctness assertions	Cross-structure invariant enforcement
Federated Topology Domain	Federation (future)	topology graph	Cross-runtime structure relationships

The semantic distinction matters: aggregation synthesizes ontology; boundary synthesizes admission topology; admissibility synthesizes execution eligibility; correctness synthesizes cross-structure invariants. Same Phase Type B mechanics — different governance concerns.

16.3 Invariants for All Federated Domains

Macro-invariant: Federated governance domains may synthesize governance products but may not synthesize execution semantics.

This is the primary protection against hidden orchestration, runtime graph mutation, and governance/execution collapse. Every domain family is subject to it without exception.

Every federated governance domain must: - be declared via an aggregation STRUCTURE_ artifact with an explicit `aggregation_type` - declare all contributing source dirs explicitly (`artifact_source_dirs`) - produce deterministic output given the same inputs - run after all contributing Phase Type A builds complete - introduce no execution node types, runtime graph primitives, or middleware chains - preserve orthogonality with the execution layer

16.4 Boundary Orthogonality (Applies to All Domains)

Federated governance products govern admissibility, routing, and observation. They may never: - mutate execution semantics - alter declared workflow behavior - inject orchestration logic - infer behavioral routing dynamically - become smart runtime components

Authority governs whether execution may exist. Transport governs where boundaries are. Neither touches how the execution graph runs.

16.5 Architectural Significance

Before	After
Protocol-governed execution	Federated governance compilation
Vocabulary is per-structure output	Vocabulary is a federated ontology product
Transport routes are implicit	Transport routing is a compiled, governed artifact
Authority is runtime middleware	Authority is a compiled admissibility domain
Governance is per-artifact	Governance is a compounding federated product

Governance surfaces compound without runtime coupling. Execution remains semantically blind. Governance grows richer. These are orthogonal trajectories — that is the architectural strength.

---

17. Quick Reference Appendix

17.1 Core Commands

```
# Bootstrap
cd scripts && ./bootstrap_pgs.sh && source .venv/bin/activate

# Run workflow
pgs_runtime run --wf <domain>::<WF_CODE> --payload <file.json> \
  --data-root /abs/path/to/pgs_workspace/data \
```

```
  --workspace /abs/path/to/pgs_workspace

# Examine trace
pgs_runtime examine pgs_workspace/traces/<TRACE_ID>/<TRACE_ID>.jsonl

# HTTP server
~/pgs_workspace/scripts/start_http_server.sh

# Build (all)
python -m pgs_compiler.cli compile --structure STRUCTURE_BUILD_PLATFORM_CONFI
G_V0
python -m pgs_compiler.cli compile --structure STRUCTURE_BUILD_BLOCKCHAIN_CON
FIG_V0
python -m pgs_compiler.cli compile --structure STRUCTURE_BUILD_AI_GOVERNANCE_
CONFIG_V0
python -m pgs_compiler.cli build --workspace /abs/path/to/pgs_workspace

# Unit tests
python pgs_runtime/testbed/run_runtime_tests.py -v

# Demo
cd scripts && ./demo_sample_workflow.sh
```

17.2 Artifact Prefix Map

```
WF_  Workflow           CC_  Capability Contract
CT_  Transform          CS_  Side Effect
IN_  Intent             RB_  Runtime Binding
EV_  Event              AC_  Actor Context
AS_  Assertion          STRUCTURE_  Build Config
TI_  Transport Ingress  TE_  Transport Egress
```

17.3 FQDN Format

```
<domain>::<ARTIFACT_CODE>_V<version>

Examples:
  blockchain::WF_REGISTER_ACTOR_UNVERIFIED_V0
  ai_governance::WF_GOVERN_AGENT_ACTION_V0
  blockchain::CC_GENERATE_ACTOR_ID_V0
  capability_transforms::CT_PURE_GENERATE_ID_V0   ← CT_ use capability_transf
orms as domain
```

Note: CT_ artifacts live in the `capability_transforms` domain (not a business domain). CS_ artifacts live in `capability_side_effects`. Both are resolved via RB_ at runtime.

Governance artifact namespace (`fb.*`): All governance artifacts (constitutions, invariants, assertions, STRUCTURE configs) use the `fb.<boundary>` namespace prefix, not a business domain:

```
fb.constitution::CONSTITUTION_FEDERATION_BOUNDARY_V0
fb.topology::CONSTITUTION_EXECUTION_TOPOLOGY_V0
fb.transport::CONSTITUTION_TRANSPORT_V0
fb.constitution::STRUCTURE_BUILD_PLATFORM_CONFIG_V0
fb.topology::STRUCTURE_RUNTIME_EXECUTION_V0
```

Short-code calls to `load_bootstrap_artifact()` (without `fb.*::` namespace) raise a hard `ValueError` — no fallback exists.

17.4 Snapshot Structure

```
protocol_snapshot/
  artifacts/
    workflows/                  ← WF_ JSON
    capability_contracts/       ← CC_ JSON
    capability_transforms/      ← CT_ JSON
    capability_side_effects/    ← CS_ JSON
    runtime_bindings/           ← RB_ JSON
    intents/                    ← IN_ JSON
    events/                     ← EV_ JSON
    actors/                     ← AC_ JSON
    layers/ invariants/ assertions/ ← governance (compiled from all 13 FB_* b
oundaries)
  visualization/<WF_NAME>/
    <WF_NAME>.graph.json        ← machine-readable DAG
    <WF_NAME>.projection.png    ← visual graph

evidence_snapshot/              ← compiler observability (written by S7, veri
fied by S8)
  <domain>/
    evidence_graph.json         ← semantic causality graph; events + edges +
families + hash
```

evidence_snapshot/ is READ-ONLY post-build — same sovereignty rule as `protocol_snapshot/`. Never edit by hand. To change the evidence graph, change the protocol source and recompile.

Filename convention: FQDN uses `::` in code but `__` (double-underscore) as the filesystem separator in artifact filenames.

```
blockchain__WF_REGISTER_ACTOR_UNVERIFIED_V0.json
```

17.5 Trace Structure

```
traces/<TRACE_ID>/
  <TRACE_ID>.jsonl    ← append-only structured event log (examine input)
  <TRACE_ID>.md       ← human-readable summary
  <TRACE_ID>.png      ← execution path visualization
```

17.6 Repo Map (One-Line)

```
pgs_governance  → govern/invariants  pgs_compiler    → compile/build
pgs_transport   → ingress/egress     pgs_runtime     → execute
pgs_capabilities→ CT/CS library     pgs_blockchain  → blockchain domain
pgs_ai_governance → AI domain       pgs_workspace   → entry point + snapshot
```

17.7 Execution Outcome Vocabulary

Intent outcomes (IN_ gate):

```
ACK             Admission accepted — graph traversal proceeds
NACK            Admission rejected — execution halted before graph entry
```

CC node outcomes:

```
SUCCESS         Normal forward completion
ALREADY_EXISTS  Idempotency guard triggered (registry)
VIOLATION       Constraint violated (governance, invariant)
DENIED          Authorization failed
BACKEND_ERROR   Infrastructure or implementation error at CS/CT boundary
```

17.8 High-Value Grep Patterns

```
# Find all workflow artifacts
ls protocol_snapshot/artifacts/workflows/

# Find binding for a specific CT
grep -r "CT_<NAME>" protocol_snapshot/artifacts/runtime_bindings/

# Find which CC uses a given CS
grep -r "CS_<NAME>" protocol_snapshot/artifacts/capability_contracts/

# Find trace events for a specific CC
grep '"artifact_id": "blockchain::CC_<NAME>"' traces/<TRACE_ID>/<TRACE_ID>.jsonl

# Check snapshot validity marker
grep "status" protocol_snapshot/artifacts/workflows/*.json | head -5
```

Architectural Stability Promise

PGS prioritizes architectural stability over feature velocity.

Backward compatibility is subordinate to governance correctness, determinism, and ontology integrity.

When architectural tradeoffs occur, preference is given to: - explicitness over convenience - governance over heuristics - determinism over flexibility - compile-time enforcement over runtime repair

---

Explicit Exclusions

This manual intentionally omits: - Implementation internals and source walkthroughs - Full protocol specifications (constitutions are authoritative) - Exhaustive command permutations (see `pgs_cli_cheatsheet.txt`) - Tutorial-style explanations - Deployment topology guidance - API reference surfaces

---

Manual Evolution Rule

New content must improve architectural cognition density.

If information is already obvious from source code or protocol artifacts, it does not belong here. If a section does not improve architectural decision quality or cognitive restoration speed, it does not belong here.

This rule is the primary protection against entropy.

End of PGS Field Manual v0 — KISS for Cognitive Processing